

Dnipro User's Guide

By Examples

Symbolic equation-based MNA for SPICE-like circuits

Version 0.1.0

Package name:	dnipro
Implementation language:	Python 3.11+
Symbolic engine:	SymPy
Optional reference simulator:	ngspice
Example root:	../examples/user_guide

May 10, 2026

Contents

1 Purpose and Scope	5
1.1 What the examples demonstrate	5
1.2 Repository layout	5
2 Conventions	5
2.1 SPICE-compatible directions	5
2.2 Unknown ordering	6
2.3 Residual, linear, and descriptor forms	6
3 Shared Python Helpers	6
4 Example 1: Voltage Divider	9
4.1 dnipro netlist	9
4.2 ngspice template	9
4.3 Python script	10
4.4 Symbolic system	10
5 Example 2: Current Source Direction	11
5.1 dnipro netlist	11
5.2 ngspice template	11
5.3 Python script	11
5.4 Symbolic system	11
6 Example 3: RC Time-Domain Descriptor System	12
6.1 dnipro netlist	12
6.2 ngspice template	12
6.3 Python script	12
6.4 Residual equations	13
6.5 Descriptor matrix	13
7 Example 4: RC Low-Pass in Laplace Domain	13
7.1 dnipro netlist	13
7.2 ngspice template	13
7.3 Python script	14
7.4 Laplace-domain matrix	14
8 Example 5: Nonlinear Diode Residual	15
8.1 dnipro netlist	15
8.2 ngspice template	15
8.3 Python script	15
8.4 Nonlinear residual	15
8.5 Expected linear extraction error	16
9 Example 6: Voltage Source Direction	16
9.1 dnipro netlist	16
9.2 ngspice template	16
9.3 Python script	17
9.4 Sign convention	17
10 Example 7: Floating Capacitor	17
10.1 dnipro netlist	17

10.2ngspice template	18
10.3Python script	18
10.4Residual structure	18
11 Example 8: RL Branch-Current Formulation	18
11.1dnipro netlist	18
11.2ngspice template	19
11.3Python script	19
11.4Inductor equation	19
12 Example 9: RL Descriptor Form	20
12.1dnipro netlist	20
12.2ngspice template	20
12.3Python script	20
12.4Descriptor contribution	20
13 Example 10: RLC Low-Pass	21
13.1dnipro netlist	21
13.2ngspice template	21
13.3Python script	21
13.4Mixed dynamic structure	22
14 Example 11: Sinusoidal Voltage Source	22
14.1dnipro netlist	22
14.2ngspice template	22
14.3Python script	22
14.4Source expression	23
15 Example 12: Sinusoidal Current Source	23
15.1dnipro netlist	23
15.2ngspice template	23
15.3Python script	24
15.4KCL contribution	24
16 Example 13: PULSE Source	24
16.1dnipro netlist	24
16.2ngspice template	24
16.3Python script	25
16.4Timing parameters	25
17 Example 14: PWL Source	25
17.1dnipro netlist	25
17.2ngspice template	25
17.3Python script	26
17.4Exact time points	26
18 Example 15: Dependent Sources	26
18.1dnipro netlist	26
18.2ngspice template	27
18.3Python script	27
18.4Device roles	27
19 Example 16: Parser RVI Details	28

19.1 dnipro netlist	28
19.2 ngspice template	28
19.3 Python script	28
19.4 Notes	29
20 Example 17: Parser C/L Values	29
20.1 dnipro netlist	29
20.2 ngspice template	29
20.3 Python script	29
20.4 Notes	30
21 Example 18: Numeric Diode Model Values	30
21.1 dnipro netlist	30
21.2 ngspice template	30
21.3 Python script	30
21.4 Exact values	30
22 Example 19: Dependent Source Parser Bundle	31
22.1 dnipro netlist	31
22.2 ngspice template	31
22.3 Python script	31
22.4 Notes	32
23 Example 20: Symbolic Diode Model Parser	32
23.1 dnipro netlist	32
23.2 ngspice template	32
23.3 Python script	32
23.4 Residual	33
24 Example 21: Single-Capacitor Frequency Substitution	33
24.1 dnipro netlist	33
24.2 ngspice template	33
24.3 Python script	33
24.4 Frequency substitution	34
25 Example 22: Missing Numeric Parameters	34
25.1 dnipro netlist	34
25.2 ngspice template	34
25.3 Python script	35
25.4 Expected error	35
26 Example 23: Linear Extraction Rejects Diode	35
26.1 dnipro netlist	35
26.2 ngspice template	35
26.3 Python script	36
26.4 Expected error	36
27 Example 24: Descriptor Rejects Laplace Domain	36
27.1 dnipro netlist	36
27.2 ngspice template	36
27.3 Python script	37
27.4 Expected error	37

28 Example 25: Case-Preserved Component Name	37
28.1 dnipro netlist	37
28.2 ngspice template	37
28.3 Python script	38
28.4 Notes	38
29 Example 26: Expected Parser Errors	38
29.1 dnipro placeholder netlist	38
29.2 ngspice placeholder template	38
29.3 Python script	39
29.4 Rejected syntax	39
A Complete Runnable Example Catalog	39
B Verification Commands	40

1 Purpose and Scope

This guide demonstrates dnipro through runnable examples. Each example is backed by files in `examples/user_guide`: a dnipro netlist, a runnable ngspice template, and a Python script that exercises the current public API.

The guide is intentionally example-driven. It is not a replacement for the reference manual. The reference manual describes the implemented API surface; this document shows how that API behaves on concrete circuits and how the resulting symbolic systems map to standard Modified Nodal Analysis notation.

1.1 What the examples demonstrate

- Residual equation construction $F(x, \dot{x}, t, p) = 0$.
- Classical linear extraction $Ax = b$ where the residuals are linear in the unknown vector.
- Time-domain descriptor extraction $Gx + C\dot{x} = b$.
- Laplace-domain linear systems $A(s)x = b$.
- Numeric and frequency substitution using exact SymPy numbers.
- Nonlinear residual construction for the diode model.
- Expected API errors for unsupported transformations.

1.2 Repository layout

Every example directory uses the same structure:

```
examples/user_guide/NN_example_name/
  dnipro.cir      # dnipro-compatible symbolic netlist
  ngspice.cir     # runnable numeric ngspice template
  run.py          # Python script using the dnipro API
```

Run a Python example from the repository root:

```
/home/konst/Documents/Projects/venv/bin/python examples/user_guide/01_voltage_divider/run.py
```

Run the matching ngspice template:

```
ngspice -b examples/user_guide/01_voltage_divider/ngspice.cir
```

2 Conventions

2.1 SPICE-compatible directions

dnipro follows the SPICE convention for voltage and current directions. For a voltage source

```
V1 nplus nminus value
```

the source voltage is $v(nplus) - v(nminus)$. The branch-current unknown i_{V1} follows the current direction through the source from the first node to the second node.

For a current source

```
I1 nplus nminus value
```

the source current is defined from the first node to the second node. In KCL residuals, dnipro writes currents leaving each node as positive. Therefore a source I_{out} contributes $+I_{in}$ to the KCL residual at out.

2.2 Unknown ordering

The unknown vector always lists node voltages first, in parser insertion order, followed by branch currents in first-appearance order. In time domain, unknowns are SymPy functions such as $v_{out}(t)$. In Laplace domain, unknowns are plain symbols such as v_{out} .

2.3 Residual, linear, and descriptor forms

The primary representation is the residual system

$$F(x, \dot{x}, t, p) = 0. \quad (1)$$

If the residuals are algebraically linear in x , dnipro can derive

$$Ax = b. \quad (2)$$

For linear time-domain systems with derivatives, dnipro can derive the descriptor form

$$Gx + C\dot{x} = b. \quad (3)$$

Classical $Ax = b$ extraction is not the right time-domain view when residuals contain derivatives such as $\frac{d}{dt}v_{out}(t)$. Use descriptor form in time domain, or use Laplace domain to obtain $A(s)x = b$.

3 Shared Python Helpers

All example scripts import the shared helper module:

```
from __future__ import annotations

# ruff: noqa: E402, I001

import sys
from pathlib import Path

import sympy as sp

REPO_ROOT = Path(__file__).resolve().parents[2]
if str(REPO_ROOT) not in sys.path:
    sys.path.insert(0, str(REPO_ROOT))

from dnipro.analysis.substitutions import ( # noqa: E402
    apply_frequency_substitution,
```

```

        apply_numeric_substitutions,
        parse_param_assignments,
    )
from dnipro.errors import DniproError # noqa: E402
from dnipro.export.text_exporter import ( # noqa: E402
    export_descriptor_text,
    export_linear_text,
    export_text,
)
from dnipro.mna.equation_builder import build_equations # noqa: E402
from dnipro.mna.linear_extraction import ( # noqa: E402
    extract_descriptor_system,
    extract_linear_system,
)
from dnipro.parser.netlist_parser import parse_netlist_file,
    parse_netlist_text # noqa: E402

def netlist_path(script_file: str, filename: str = "dnipro.cir") -> Path:
    return Path(script_file).resolve().with_name(filename)

def build_system(
    path: Path,
    *,
    domain: str = "time",
    params: list[str] | None = None,
    freq_hz: sp.Expr | None = None,
    omega: sp.Expr | None = None,
):
    circuit = parse_netlist_file(path)
    system = build_equations(circuit, domain=domain)

    if freq_hz is not None or omega is not None:
        system = apply_frequency_substitution(system, freq_hz=freq_hz,
        omega=omega)

    if params:
        system = apply_numeric_substitutions(system,
        parse_param_assignments(params))

    return system

def print_section(title: str) -> None:
    print(f"\n=== {title} ===")

def print_parsed_circuit(path: Path) -> None:
    print_section("Parsed circuit")
    circuit = parse_netlist_file(path)
    print(f"nodes: {'', ' '.join(circuit.node_order) or '(none)'}")
    print("components:")
    for component in circuit.components:
        print(f" {component!r}")
    if circuit.models:
        print("models:")
        for name, model in circuit.models.items():

```

```
        print(f" {name}: {model!r}")

def print_residuals(
    path: Path,
    *,
    domain: str = "time",
    params: list[str] | None = None,
    freq_hz: sp.Expr | None = None,
    omega: sp.Expr | None = None,
) -> None:
    print_section(f"Residual system ({domain})")
    system = build_system(
        path,
        domain=domain,
        params=params,
        freq_hz=freq_hz,
        omega=omega,
    )
    print(export_text(system))

def print_linear(
    path: Path,
    *,
    domain: str = "time",
    params: list[str] | None = None,
    freq_hz: sp.Expr | None = None,
    omega: sp.Expr | None = None,
) -> None:
    print_section(f"Classical linear MNA ({domain})")
    system = build_system(
        path,
        domain=domain,
        params=params,
        freq_hz=freq_hz,
        omega=omega,
    )
    print(export_linear_text(extract_linear_system(system)))

def print_descriptor(path: Path, *, params: list[str] | None = None) -> None:
    print_section("Descriptor MNA (time)")
    system = build_system(path, domain="time", params=params)
    print(export_descriptor_text(extract_descriptor_system(system)))

def print_linear_error(path: Path, *, domain: str = "time") -> None:
    print_section("Expected linear extraction error")
    system = build_system(path, domain=domain)
    try:
        extract_linear_system(system)
    except DniproError as error:
        print(f"{type(error).__name__}: {error}")
    else:
        raise AssertionError("linear extraction unexpectedly succeeded")
```

```

def print_descriptor_error(path: Path, *, domain: str = "laplace") -> None:
    print_section("Expected descriptor extraction error")
    system = build_system(path, domain=domain)
    try:
        extract_descriptor_system(system)
    except DniproError as error:
        print(f"{type(error).__name__}: {error}")
    else:
        raise AssertionError("descriptor extraction unexpectedly succeeded")
)

def print_parse_errors(cases: list[str]) -> None:
    print_section("Expected parse errors")
    for case in cases:
        try:
            parse_netlist_text(case)
        except DniproError as error:
            print(f"{case!r} -> {type(error).__name__}: {error}")
        else:
            raise AssertionError(f"parse unexpectedly succeeded: {case!r}")

```

The helpers are deliberately thin wrappers around the public dnipro API:

- `parse_netlist_file`;
- `build_equations`;
- `extract_linear_system`;
- `extract_descriptor_system`;
- the text exporters.

4 Example 1: Voltage Divider

This example is the smallest useful linear circuit. It demonstrates resistor KCL, voltage-source branch current allocation, residual reporting, and classical $Ax = b$ extraction.

4.1 dnipro netlist

```

V1 in 0 Vin
R1 in out R1
R2 out 0 R2

```

4.2 ngspice template

```

* Voltage divider operating-point example.
.param Vin=10 R1=1k R2=1k
V1 in 0 DC {Vin}
R1 in out {R1}
R2 out 0 {R2}
.op
.control
op
print v(in) v(out) i(v1)

```

```
quit
.endc
.end
```

4.3 Python script

```
from __future__ import annotations

# ruff: noqa: E402, I001

import sys
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parents[1]))

from common import netlist_path, print_linear, print_residuals

NETLIST = netlist_path(__file__)

print_residuals(NETLIST)
print_linear(NETLIST)
print_linear(NETLIST, params=["Vin=10", "R1=1k", "R2=1k"])
```

4.4 Symbolic system

The unknown vector is

$$x = \begin{bmatrix} v_{in}(t) \\ v_{out}(t) \\ i_{V1}(t) \end{bmatrix}. \quad (4)$$

The residual equations are

$$\frac{v_{in} - v_{out}}{R_1} + i_{V1} = 0, \quad (5)$$

$$\frac{v_{out} - v_{in}}{R_1} + \frac{v_{out}}{R_2} = 0, \quad (6)$$

$$v_{in} - V_{in} = 0. \quad (7)$$

Since the system is linear in x , dnipro extracts

$$A = \begin{bmatrix} \frac{1}{R_1} & -\frac{1}{R_1} & 1 \\ -\frac{1}{R_1} & \frac{1}{R_1} + \frac{1}{R_2} & 0 \\ 1 & 0 & 0 \end{bmatrix}, \quad b = \begin{bmatrix} 0 \\ 0 \\ V_{in} \end{bmatrix}. \quad (8)$$

With $V_{in} = 10$, $R_1 = R_2 = 1 \text{ k}\Omega$, the ngspice template reports $v(out) = 5 \text{ V}$. The sign of $i(v1)$ follows ngspice's source-current convention.

5 Example 2: Current Source Direction

This example isolates the SPICE current-source direction convention.

5.1 dnipro netlist

```
I1 out 0 Iin
R1 out 0 R
```

5.2 ngspice template

```
* Current source direction example.
* I1 current flows from out to ground, matching SPICE source direction.
.param Iin=1m R=1k
I1 out 0 DC {Iin}
R1 out 0 {R}
.op
.control
op
print v(out)
quit
.endc
.end
```

5.3 Python script

```
from __future__ import annotations

# ruff: noqa: E402, I001

import sys
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parents[1]))

from common import netlist_path, print_linear, print_residuals

NETLIST = netlist_path(__file__)

print_residuals(NETLIST)
print_linear(NETLIST)
print_linear(NETLIST, params=["Iin=1m", "R=1k"])
```

5.4 Symbolic system

The source I1 out 0 Iin defines current from out to ground. Because dnipro writes KCL as a sum of currents leaving the node, the residual is

$$I_{in} + \frac{v_{out}}{R} = 0. \quad (9)$$

The extracted scalar system is

$$A = \left[\frac{1}{R} \right], \quad x = [v_{out}(t)], \quad b = [-I_{in}]. \quad (10)$$

For $I_{in} = 1 \text{ mA}$ and $R = 1 \text{ k}\Omega$, the operating point is $v(out) = -1 \text{ V}$. This negative voltage is expected: a positive current source from out to ground removes current from the node.

6 Example 3: RC Time-Domain Descriptor System

This example introduces a capacitor and therefore a time derivative. The primary time-domain report is the residual system, and the matrix view is the descriptor form $Gx + C\dot{x} = b$.

6.1 dnipro netlist

```
V1 in 0 Vin
R1 in out R
C1 out 0 C
```

6.2 ngspice template

```
* RC low-pass transient template.
.param Vin=5 R=1k C=1u
V1 in 0 DC {Vin}
R1 in out {R}
C1 out 0 {C}
.op
.control
op
print v(in) v(out) i(v1)
quit
.endc
.end
```

6.3 Python script

```
from __future__ import annotations

# ruff: noqa: E402, I001

import sys
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parents[1]))

from common import netlist_path, print_descriptor, print_residuals

NETLIST = netlist_path(__file__)
PARAMS = ["Vin=5", "R=1k", "C=1u"]

print_residuals(NETLIST)
print_descriptor(NETLIST)
print_residuals(NETLIST, params=PARAMS)
```

6.4 Residual equations

The unknown vector is

$$x = \begin{bmatrix} v_{in}(t) \\ v_{out}(t) \\ i_{V1}(t) \end{bmatrix}, \quad \dot{x} = \begin{bmatrix} \dot{v}_{in}(t) \\ \dot{v}_{out}(t) \\ \dot{i}_{V1}(t) \end{bmatrix}. \quad (11)$$

The residual equations are

$$\frac{v_{in} - v_{out}}{R} + i_{V1} = 0, \quad (12)$$

$$\frac{v_{out} - v_{in}}{R} + C \frac{dv_{out}}{dt} = 0, \quad (13)$$

$$v_{in} - V_{in} = 0. \quad (14)$$

6.5 Descriptor matrix

dnipro extracts

$$G = \begin{bmatrix} \frac{1}{R} & -\frac{1}{R} & 1 \\ -\frac{1}{R} & \frac{1}{R} & 0 \\ 1 & 0 & 0 \end{bmatrix}, \quad C = \begin{bmatrix} 0 & 0 & 0 \\ 0 & C & 0 \\ 0 & 0 & 0 \end{bmatrix}, \quad b = \begin{bmatrix} 0 \\ 0 \\ V_{in} \end{bmatrix}. \quad (15)$$

The nonzero $C_{2,2}$ entry comes from the capacitor current $C dv_{out}/dt$. Classical $Ax = b$ extraction is not used here because the time-domain residuals contain derivatives.

7 Example 4: RC Low-Pass in Laplace Domain

The same RC circuit can be built in Laplace domain. In that mode the capacitor contribution becomes the algebraic admittance sC , and the circuit has a classical linear system $A(s)x = b$.

7.1 dnipro netlist

```
V1 in 0 Vin
R1 in out R
C1 out 0 C
```

7.2 ngspice template

```
* RC low-pass AC template.
.param Vin=1 R=1k C=1u
V1 in 0 AC {Vin}
R1 in out {R}
C1 out 0 {C}
.ac dec 20 1 1Meg
.control
run
print v(out)
quit
```

```
.endc
.end
```

7.3 Python script

```
from __future__ import annotations

# ruff: noqa: E402, I001

import sys
from pathlib import Path

import sympy as sp

sys.path.insert(0, str(Path(__file__).resolve().parents[1]))

from common import netlist_path, print_linear, print_residuals

NETLIST = netlist_path(__file__)

print_residuals(NETLIST, domain="laplace")
print_linear(NETLIST, domain="laplace")
print_residuals(NETLIST, domain="laplace", omega=sp.Integer(1000))
print_residuals(NETLIST, domain="laplace", freq_hz=sp.Integer(50))
```

7.4 Laplace-domain matrix

The unknown vector is

$$x = \begin{bmatrix} v_{in} \\ v_{out} \\ i_{V1} \end{bmatrix}. \quad (16)$$

The extracted matrix is

$$A(s) = \begin{bmatrix} \frac{1}{R} & -\frac{1}{R} & 1 \\ -\frac{1}{R} & \frac{1}{R} + sC & 0 \\ 1 & 0 & 0 \end{bmatrix}, \quad b = \begin{bmatrix} 0 \\ 0 \\ V_{in} \end{bmatrix}. \quad (17)$$

The example script also demonstrates frequency substitution:

$$s = j\omega, \quad (18)$$

$$s = j2\pi f. \quad (19)$$

For `--freq 50`, dnipro records $s = 100j\pi$ in the residual system metadata.

8 Example 5: Nonlinear Diode Residual

This example demonstrates the nonlinear diode component. The circuit is symbolically supported as a residual system, but it is intentionally rejected by classical linear extraction.

8.1 dnipro netlist

```
V1 in 0 Vin
R1 in out R
D1 out 0 Ddefault
.model Ddefault D(Is=Is n=n Vt=Vt)
```

8.2 ngspice template

```
* Nonlinear diode operating-point template.
.param Vin=5 R=1k
V1 in 0 DC {Vin}
R1 in out {R}
D1 out 0 Ddefault
.model Ddefault D(Is=1e-14 n=1)
.op
.control
op
print v(in) v(out) i(v1)
quit
.endc
.end
```

8.3 Python script

```
from __future__ import annotations

# ruff: noqa: E402, I001

import sys
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parents[1]))

from common import netlist_path, print_linear_error, print_residuals

NETLIST = netlist_path(__file__)
PARAMS = ["Vin=5", "R=1k", "Is=1e-14", "n=1", "Vt=25.85m"]

print_residuals(NETLIST)
print_residuals(NETLIST, params=PARAMS)
print_linear_error(NETLIST)
```

8.4 Nonlinear residual

The unknown vector is

$$x = \begin{bmatrix} v_{in}(t) \\ v_{out}(t) \\ i_{V1}(t) \end{bmatrix}. \quad (20)$$

The diode current is

$$i_D = I_s \left(\exp\left(\frac{v_{out}}{nV_t}\right) - 1 \right). \quad (21)$$

The residual equations are

$$\frac{v_{in} - v_{out}}{R} + i_{V1} = 0, \quad (22)$$

$$\frac{v_{out} - v_{in}}{R} + I_s \left(\exp\left(\frac{v_{out}}{nV_t}\right) - 1 \right) = 0, \quad (23)$$

$$v_{in} - V_{in} = 0. \quad (24)$$

8.5 Expected linear extraction error

This residual system is nonlinear in v_{out} . Calling `extract_linear_system` therefore raises `NonlinearSystemError`. This is expected behavior: `dnipro` has built the correct symbolic nonlinear system, but it does not pretend that the system can be represented as a classical linear $Ax = b$ matrix.

The example script also applies exact numeric substitutions:

```
Vin=5, R=1k, Is=1e-14, n=1, Vt=25.85m
```

These values are parsed into exact SymPy numbers before substitution.

9 Example 6: Voltage Source Direction

The previous examples used a voltage source from a named node to ground. This example reverses the source terminals and demonstrates the resulting sign in the voltage constraint.

9.1 dnipro netlist

```
V1 0 out Vin
R1 out 0 R
```

9.2 ngspice template

```
* Voltage source direction example.
.param Vin=5 R=1k
V1 0 out DC {Vin}
R1 out 0 {R}
.op
.control
op
```

```
print v(out) i(v1)
quit
.endc
.end
```

9.3 Python script

```
from __future__ import annotations
# ruff: noqa: E402, I001

import sys
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parents[1]))

from common import netlist_path, print_linear, print_residuals

NETLIST = netlist_path(__file__)

print_residuals(NETLIST)
print_linear(NETLIST)
print_linear(NETLIST, params=["Vin=5", "R=1k"])
```

9.4 Sign convention

For V1 0 out Vin, the source voltage is $v(0) - v(out)$. Since ground is zero, the voltage constraint is

$$-v_{out} - V_{in} = 0. \quad (25)$$

The KCL equation at out is

$$\frac{v_{out}}{R} - i_{V1} = 0, \quad (26)$$

because the branch current i_{V1} is defined from ground to out. With $V_{in} = 5$, ngspice reports $v(out) = -5\text{ V}$, which is exactly the source orientation encoded by the netlist.

10 Example 7: Floating Capacitor

This example places the capacitor between two non-ground nodes. It demonstrates that the capacitor derivative contributes to both KCL equations with opposite signs.

10.1 dnipro netlist

```
V1 in 0 Vin
C1 in out C
R1 out 0 R
```

10.2 ngspice template

```
* Floating capacitor transient template.
.param Vin=5 C=1u R=1k
V1 in 0 DC {Vin}
C1 in out {C}
R1 out 0 {R}
.op
.control
op
print v(in) v(out) i(v1)
quit
.endc
.end
```

10.3 Python script

```
from __future__ import annotations

# ruff: noqa: E402, I001

import sys
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parents[1]))

from common import netlist_path, print_descriptor, print_residuals

NETLIST = netlist_path(__file__)

print_residuals(NETLIST)
print_descriptor(NETLIST)
```

10.4 Residual structure

The key capacitor current is

$$i_C = C \frac{d}{dt} (v_{in} - v_{out}). \quad (27)$$

Therefore the two KCL residuals contain $+i_C$ at in and $-i_C$ at out. This is a useful conservation check for dynamic two-terminal elements.

11 Example 8: RL Branch-Current Formulation

Inductors are represented with branch-current unknowns in both time and Laplace domains. This keeps the formulation aligned with SPICE-style MNA.

11.1 dnipro netlist

```
V1 in 0 Vin
L1 in out L
R1 out 0 R
```

11.2 ngspice template

```
* RL low-pass AC template.
.param Vin=1 L=10m R=100
V1 in 0 AC {Vin}
L1 in out {L}
R1 out 0 {R}
.ac dec 20 1 1Meg
.control
run
print v(out) i(l1)
quit
.endc
.end
```

11.3 Python script

```
from __future__ import annotations

# ruff: noqa: E402, I001

import sys
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parents[1]))

from common import netlist_path, print_descriptor, print_linear,
    print_residuals

NETLIST = netlist_path(__file__)

print_residuals(NETLIST)
print_descriptor(NETLIST)
print_residuals(NETLIST, domain="laplace")
print_linear(NETLIST, domain="laplace")
```

11.4 Inductor equation

The time-domain branch equation is

$$v_{in} - v_{out} - L \frac{di_{L1}}{dt} = 0. \quad (28)$$

In Laplace domain the same relation becomes

$$v_{in} - v_{out} - sL i_{L1} = 0. \quad (29)$$

The branch current i_{L1} is part of the unknown vector. This is the same principle used for voltage sources and controlled voltage sources.

12 Example 9: RL Descriptor Form

This variant connects the inductor from out to ground and shows how the inductor derivative appears in the descriptor matrix.

12.1 dnipro netlist

```
V1 in 0 Vin
R1 in out R
L1 out 0 L
```

12.2 ngspice template

```
* RL descriptor example with inductor to ground.
.param Vin=1 R=100 L=10m
V1 in 0 AC {Vin}
R1 in out {R}
L1 out 0 {L}
.ac dec 20 1 1Meg
.control
run
print v(out) i(l1)
quit
.endc
.end
```

12.3 Python script

```
from __future__ import annotations

# ruff: noqa: E402, I001

import sys
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parents[1]))

from common import netlist_path, print_descriptor, print_linear,
    print_residuals

NETLIST = netlist_path(__file__)

print_residuals(NETLIST)
print_descriptor(NETLIST)
print_residuals(NETLIST, domain="laplace")
print_linear(NETLIST, domain="laplace")
```

12.4 Descriptor contribution

For the unknown vector

$$x = \begin{bmatrix} v_{in}(t) \\ v_{out}(t) \\ i_{V1}(t) \\ i_{L1}(t) \end{bmatrix}, \quad (30)$$

the inductor equation contributes a dynamic coefficient $-L$ in the row for L1 and the column for i_{L1} . The descriptor form is therefore the natural time-domain matrix representation for this circuit.

13 Example 10: RLC Low-Pass

This example combines a resistor, an inductor branch current, and a capacitor voltage derivative in one circuit.

13.1 dnipro netlist

```
V1 in 0 Vin
R1 in n1 R
L1 n1 out L
C1 out 0 C
```

13.2 ngspice template

```
* RLC low-pass AC template.
.param Vin=1 R=100 L=10m C=1u
V1 in 0 AC {Vin}
R1 in n1 {R}
L1 n1 out {L}
C1 out 0 {C}
.ac dec 20 1 1Meg
.control
run
print v(out) i(l1)
quit
.endc
.end
```

13.3 Python script

```
from __future__ import annotations

# ruff: noqa: E402, I001

import sys
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parents[1]))

from common import netlist_path, print_descriptor, print_linear,
    print_residuals
```

```
NETLIST = netlist_path(__file__)

print_residuals(NETLIST)
print_descriptor(NETLIST)
print_residuals(NETLIST, domain="laplace")
print_linear(NETLIST, domain="laplace")
```

13.4 Mixed dynamic structure

The residual system contains both dynamic patterns:

- the capacitor contributes $C dv_{out}/dt$ to the KCL equation at out;
- the inductor contributes the branch equation $v_{n1} - v_{out} - L di_{L1}/dt = 0$.

In time domain, use descriptor extraction. In Laplace domain, the same example has a classical $A(s)x = b$ matrix.

14 Example 11: Sinusoidal Voltage Source

This example shows a symbolic sinusoidal voltage source in time domain.

14.1 dnipro netlist

```
V1 in 0 SIN(0 Vm omega)
R1 in out R
C1 out 0 C
```

14.2 ngspice template

```
* Sinusoidal voltage-source transient template.
.param Vm=1 omega=6283.185307 R=1k C=1u
V1 in 0 SIN(0 {Vm} {omega})
R1 in out {R}
C1 out 0 {C}
.tran 10u 5m
.control
run
print v(in) v(out) i(v1)
quit
.endc
.end
```

14.3 Python script

```
from __future__ import annotations

# ruff: noqa: E402, I001

import sys
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parents[1]))
```

```

from common import netlist_path, print_residuals

NETLIST = netlist_path(__file__)

print_residuals(NETLIST)
print_residuals(NETLIST, params=["Vm=1", "omega=6283.185307", "R=1k", "C=1u
    "])

```

14.4 Source expression

The source

```
SIN(0 Vm omega)
```

is represented as

$$V_m \sin(\omega t). \quad (31)$$

The voltage-source residual is therefore

$$v_{in}(t) - V_m \sin(\omega t) = 0. \quad (32)$$

15 Example 12: Sinusoidal Current Source

The current-source version demonstrates that the same source expression is inserted into KCL using SPICE current direction.

15.1 dnipro netlist

```

I1 out 0 SIN(0 Im omega)
R1 out 0 R
C1 out 0 C

```

15.2 ngspice template

```

* Sinusoidal current-source transient template.
.param Im=1m omega=6283.185307 R=1k C=1u
I1 out 0 SIN(0 {Im} {omega})
R1 out 0 {R}
C1 out 0 {C}
.tran 10u 5m
.control
run
print v(out)
quit
.endc
.end

```

15.3 Python script

```
from __future__ import annotations

# ruff: noqa: E402, I001

import sys
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parents[1]))

from common import netlist_path, print_residuals

NETLIST = netlist_path(__file__)

print_residuals(NETLIST)
print_residuals(NETLIST, params=["Im=1m", "omega=6283.185307", "R=1k", "C=1u"])
```

15.4 KCL contribution

For I_l out $\theta \sin(\theta \text{ Im } \omega t)$, the KCL residual at out includes

$$I_m \sin(\omega t) + \frac{v_{out}}{R} + C \frac{dv_{out}}{dt} = 0. \quad (33)$$

The positive sign on the source term follows directly from the current direction from out to ground.

16 Example 13: PULSE Source

The PULSE source is expanded into a symbolic SymPy Piecewise expression.

16.1 dnipro netlist

```
V1 in 0 PULSE(0 Vh 0 tr tf ton T)
R1 in out R
C1 out 0 C
```

16.2 ngspice template

```
* PULSE voltage-source transient template.
.param Vh=5 tr=1u tf=1u ton=1m T=2m R=1k C=1u
V1 in 0 PULSE(0 {Vh} 0 {tr} {tf} {ton} {T})
R1 in out {R}
C1 out 0 {C}
.tran 10u 5m
.control
run
print v(in) v(out) i(v1)
quit
.endc
.end
```

16.3 Python script

```

from __future__ import annotations

# ruff: noqa: E402, I001

import sys
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parents[1]))

from common import netlist_path, print_residuals

NETLIST = netlist_path(__file__)

print_residuals(NETLIST)
print_residuals(
    NETLIST,
    params=["Vh=5", "tr=1u", "tf=1u", "ton=1m", "T=2m", "R=1k", "C=1u"],
)

```

16.4 Timing parameters

The example uses the supported seven-argument source form:

```
PULSE(0 Vh 0 tr tf ton T)
```

The duty-related timing is explicit: rise time, fall time, on time, and period are all supplied. dnipro keeps those symbols exact and builds the resulting piecewise source expression.

17 Example 14: PWL Source

Piecewise-linear sources are also converted to SymPy Piecewise expressions.

17.1 dnipro netlist

```

V1 in 0 PWL(0 0 1m 5 2m 0)
R1 in out R
C1 out 0 C

```

17.2 ngspice template

```

* PWL voltage-source transient template.
.param R=1k C=1u
V1 in 0 PWL(0 0 1m 5 2m 0)
R1 in out {R}
C1 out 0 {C}
.tran 10u 5m
.control
run
print v(in) v(out) i(v1)
quit

```

```
.endc
.end
```

17.3 Python script

```
from __future__ import annotations

# ruff: noqa: E402, I001

import sys
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parents[1]))

from common import netlist_path, print_residuals

NETLIST = netlist_path(__file__)

print_residuals(NETLIST)
print_residuals(NETLIST, params=["R=1k", "C=1u"])
```

17.4 Exact time points

The source

```
PWL(0 0 1m 5 2m 0)
```

uses exact rational time values internally: $1\text{ ms} = 1/1000$ and $2\text{ ms} = 1/500$. This is important for stable symbolic output and JSON serialization.

18 Example 15: Dependent Sources

This example combines all four dependent source families in one circuit: VCVS, VCCS, CCCS, and CCVS.

18.1 dnipro netlist

```
V1 in 0 Vin
R1 in sense R1
Vsense sense 0 0
E1 evout 0 in 0 mu
G1 gvout 0 in 0 gm
F1 fvout 0 Vsense beta
H1 hvout 0 Vsense rm
Re evout 0 Rload
Rg gvout 0 Rload
Rf fvout 0 Rload
Rh hvout 0 Rload
```

18.2 ngspice template

```
* Dependent-source operating-point template.
.param Vin=1 Rl=1k Rload=10k mu=10 gm=1m beta=5 rm=100
V1 in 0 DC {Vin}
R1 in sense {Rl}
Vsense sense 0 DC 0
E1 evout 0 in 0 {mu}
G1 gvout 0 in 0 {gm}
F1 fvout 0 Vsense {beta}
H1 hvout 0 Vsense {rm}
Re evout 0 {Rload}
Rg gvout 0 {Rload}
Rf fvout 0 {Rload}
Rh hvout 0 {Rload}
.op
.control
op
print v(in) v(sense) v(evout) v(gvout) v(fvout) v(hvout)
quit
.endc
.end
```

18.3 Python script

```
from __future__ import annotations

# ruff: noqa: E402, I001

import sys
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parents[1]))

from common import netlist_path, print_linear, print_residuals

NETLIST = netlist_path(__file__)

print_residuals(NETLIST)
print_linear(NETLIST)
```

18.4 Device roles

Type	Example	Role
VCVS	E1	Adds its own branch current and a controlled voltage constraint.
VCCS	G1	Adds a controlled current contribution to KCL without a new branch current.
CCCS	F1	Uses the existing current through Vsense as the control current.
CCVS	H1	Adds its own branch current and uses Vsense as the control current.

Current-controlled sources require a control branch. The examples use a zero-volt voltage source V_{sense} , which is the standard SPICE technique for measuring a branch current.

19 Example 16: Parser RVI Details

This example demonstrates parser behavior for full-line comments, ground aliases, numeric values, and `DC(...)` source syntax.

19.1 dnipro netlist

```
* full-line comments are ignored
V1 in 0 Vin
R1 in out 1k
Iload out gnd DC(Iload)
```

19.2 ngspice template

```
* Parser-compatible R/V/I example with comment and gnd alias.
.param Vin=5 Iload=1m
V1 in 0 DC {Vin}
R1 in out 1k
Iload out gnd DC {Iload}
.op
.control
op
print v(in) v(out)
quit
.endc
.end
```

19.3 Python script

```
from __future__ import annotations

# ruff: noqa: E402, I001

import sys
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parents[1]))

from common import netlist_path, print_parsed_circuit, print_residuals

NETLIST = netlist_path(__file__)

print_parsed_circuit(NETLIST)
print_residuals(NETLIST)
```

19.4 Notes

The parser ignores full-line comments beginning with *. Ground aliases such as gnd normalize to the internal ground node. The resistor value 1k is parsed as the exact integer 1000, while DC(Iload) is reduced to the symbolic source value *Iload*.

20 Example 17: Parser C/L Values

This parser-oriented example checks capacitor and inductor lines.

20.1 dnipro netlist

```
C1 in out 1u
L1 out 0 Lload
```

20.2 ngspice template

```
* Parser-compatible C/L value example.
.param Lload=10m
Itest in 0 AC 1
C1 in out 1u
L1 out 0 {Lload}
Rleak in 0 1G
.ac dec 20 1 1Meg
.control
run
print v(in) v(out) i(l1)
quit
.endc
.end
```

20.3 Python script

```
from __future__ import annotations

# ruff: noqa: E402, I001

import sys
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parents[1]))

from common import netlist_path, print_parsed_circuit, print_residuals

NETLIST = netlist_path(__file__)

print_parsed_circuit(NETLIST)
print_residuals(NETLIST)
print_residuals(NETLIST, domain="laplace")
```

20.4 Notes

The dnipro netlist is intentionally minimal because it mirrors the parser unit test. The ngspice wrapper adds an excitation source and a large leak resistor so that the external simulator has a nonsingular operating point before AC analysis. The symbolic dnipro example does not need that wrapper element.

21 Example 18: Numeric Diode Model Values

This example focuses on diode model parsing rather than circuit solving.

21.1 dnipro netlist

```
.model Ddefault D(Is=1e-14 n=1 Vt=25.85m)
```

21.2 ngspice template

```
* Numeric diode model values in a runnable operating-point template.
.param Vin=1 R=1k
V1 in 0 DC {Vin}
R1 in out {R}
D1 out 0 Ddefault
.model Ddefault D(Is=1e-14 n=1)
.op
.control
op
print v(in) v(out) i(v1)
quit
.endc
.end
```

21.3 Python script

```
from __future__ import annotations

# ruff: noqa: E402, I001

import sys
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parents[1]))

from common import netlist_path, print_parsed_circuit

NETLIST = netlist_path(__file__)

print_parsed_circuit(NETLIST)
```

21.4 Exact values

The parser converts the model

```
.model Ddefault D(Is=1e-14 n=1 Vt=25.85m)
```

into exact SymPy numbers:

$$I_s = \frac{1}{10^{14}}, \quad n = 1, \quad V_t = \frac{517}{20000}. \quad (34)$$

22 Example 19: Dependent Source Parser Bundle

This parser bundle contains all four dependent source types without the additional elements needed for a complete circuit solve.

22.1 dnipro netlist

```
E1 out 0 in 0 mu
G1 out 0 in ref gm
F1 out 0 Vsense beta
H1 out 0 Vsense rm
```

22.2 ngspice template

```
* Dependent source parser bundle in a runnable ngspice wrapper.
.param Vin=1 mu=2 gm=1m beta=3 rm=100 Rload=1k
V1 in 0 DC {Vin}
Vref ref 0 DC 0.5
Vsense sense 0 DC 0
E1 eout 0 in 0 {mu}
G1 gout 0 in ref {gm}
F1 fout 0 Vsense {beta}
H1 hout 0 Vsense {rm}
Re eout 0 {Rload}
Rg gout 0 {Rload}
Rf fout 0 {Rload}
Rh hout 0 {Rload}
.op
.control
op
print v(eout) v(gout) v(fout) v(hout)
quit
.endc
.end
```

22.3 Python script

```
from __future__ import annotations

# ruff: noqa: E402, I001

import sys
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parents[1]))
```

```
from common import netlist_path, print_parsed_circuit

NETLIST = netlist_path(__file__)

print_parsed_circuit(NETLIST)
```

22.4 Notes

The dnipro netlist demonstrates typed parsing only. The ngspice wrapper adds independent sources and loads so the dependent devices can be evaluated in a runnable operating-point analysis.

23 Example 20: Symbolic Diode Model Parser

This example contains a diode and symbolic diode model without the surrounding source and resistor used in the full nonlinear example.

23.1 dnipro netlist

```
D1 out 0 Ddefault
.model Ddefault D(Is=Is n=n Vt=Vt)
```

23.2 ngspice template

```
* Symbolic diode parser example in a runnable ngspice wrapper.
.param Vin=1 R=1k
V1 in 0 DC {Vin}
R1 in out {R}
D1 out 0 Ddefault
.model Ddefault D(Is=1e-14 n=1)
.op
.control
op
print v(in) v(out) i(v1)
quit
.endc
.end
```

23.3 Python script

```
from __future__ import annotations

# ruff: noqa: E402, I001

import sys
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parents[1]))

from common import netlist_path, print_parsed_circuit, print_residuals
```

```
NETLIST = netlist_path(__file__)
print_parsed_circuit(NETLIST)
print_residuals(NETLIST)
```

23.4 Residual

With only the diode connected from out to ground, the KCL residual is the diode current itself:

$$I_s \left(\exp\left(\frac{v_{out}}{nV_t}\right) - 1 \right) = 0. \quad (35)$$

24 Example 21: Single-Capacitor Frequency Substitution

This example isolates the Laplace-domain capacitor term and frequency substitution machinery.

24.1 dnipro netlist

```
C1 out 0 C
```

24.2 ngspice template

```
* Single-capacitor AC template with a current excitation.
.param C=1u
Itest out 0 AC 1
C1 out 0 {C}
Rleak out 0 1G
.ac dec 20 1 1Meg
.control
run
print v(out)
quit
.endc
.end
```

24.3 Python script

```
from __future__ import annotations

# ruff: noqa: E402, I001

import sys
from pathlib import Path

import sympy as sp

sys.path.insert(0, str(Path(__file__).resolve().parents[1]))
```

```

from common import netlist_path, print_residuals

NETLIST = netlist_path(__file__)

print_residuals(NETLIST)
print_residuals(NETLIST, domain="laplace")
print_residuals(NETLIST, domain="laplace", freq_hz=sp.Integer(50))

```

24.4 Frequency substitution

In Laplace domain the residual is

$$sC v_{out} = 0. \quad (36)$$

For $f = 50$ Hz, dnipro substitutes

$$s = j2\pi f = 100j\pi. \quad (37)$$

The ngspice wrapper adds a current excitation and a large leak resistor only to make the AC analysis runnable.

25 Example 22: Missing Numeric Parameters

Numeric substitution is strict: every symbolic parameter present in the residual system must be provided.

25.1 dnipro netlist

```

V1 in 0 Vin
R1 in out R
C1 out 0 C

```

25.2 ngspice template

```

* Complete numeric ngspice version of the missing-parameter example.
.param Vin=5 R=1k C=1u
V1 in 0 DC {Vin}
R1 in out {R}
C1 out 0 {C}
.op
.control
op
print v(in) v(out) i(v1)
quit
.endc
.end

```

25.3 Python script

```

from __future__ import annotations

# ruff: noqa: E402, I001

import sys
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parents[1]))

from common import build_system, netlist_path, print_section
from dnipro.analysis.substitutions import apply_numeric_substitutions
from dnipro.errors import DniproError

NETLIST = netlist_path(__file__)

print_section("Expected missing-parameter error")
system = build_system(NETLIST)
try:
    apply_numeric_substitutions(system, {})
except DniproError as error:
    print(f"{type(error).__name__}: {error}")
else:
    raise AssertionError("numeric substitution unexpectedly succeeded")

```

25.4 Expected error

The script intentionally calls `apply_numeric_substitutions` with no parameters. The expected result is `MissingParameterError`, reporting C, R, and Vin.

26 Example 23: Linear Extraction Rejects Diode

This example repeats the diode circuit but focuses only on the expected failure mode of classical linear extraction.

26.1 dnipro netlist

```

V1 in 0 Vin
R1 in out R
D1 out 0 Ddefault
.model Ddefault D(Is=Is n=n Vt=Vt)

```

26.2 ngspice template

```

* Nonlinear diode example used to demonstrate linear extraction rejection.
.param Vin=5 R=1k
V1 in 0 DC {Vin}
R1 in out {R}
D1 out 0 Ddefault
.model Ddefault D(Is=1e-14 n=1)
.op
.control

```

```

op
print v(in) v(out) i(v1)
quit
.endc
.end

```

26.3 Python script

```

from __future__ import annotations

# ruff: noqa: E402, I001

import sys
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parents[1]))

from common import netlist_path, print_linear_error, print_residuals

NETLIST = netlist_path(__file__)

print_residuals(NETLIST)
print_linear_error(NETLIST)

```

26.4 Expected error

The residual system is valid but nonlinear. Calling `extract_linear_system` raises `NonlinearSystemError`. This explicit failure prevents accidental use of a linear matrix representation for a nonlinear circuit.

27 Example 24: Descriptor Rejects Laplace Domain

Descriptor extraction is defined for time-domain systems only.

27.1 dnipro netlist

```
C1 out 0 C
```

27.2 ngspice template

```

* Single capacitor AC template for Laplace-domain descriptor rejection
  example.
.param C=1u
Itest out 0 AC 1
C1 out 0 {C}
Rleak out 0 1G
.ac dec 20 1 1Meg
.control
run
print v(out)
quit
.endc

```

```
.end
```

27.3 Python script

```
from __future__ import annotations

# ruff: noqa: E402, I001

import sys
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parents[1]))

from common import netlist_path, print_descriptor_error, print_residuals

NETLIST = netlist_path(__file__)

print_residuals(NETLIST, domain="laplace")
print_descriptor_error(NETLIST, domain="laplace")
```

27.4 Expected error

The Laplace-domain residual $sC v_{out} = 0$ is algebraic. Requesting descriptor extraction from that system raises `NonlinearSystemError` with a message explaining that descriptor form is available only for time-domain systems.

28 Example 25: Case-Preserved Component Name

Component type dispatch is case-insensitive, but the original component name is preserved.

28.1 dnipro netlist

```
r_load in out Rload
```

28.2 ngspice template

```
* Lowercase resistor name example in a runnable wrapper.
.param Vin=1 Rload=1k
V1 in 0 DC {Vin}
r_load in out {Rload}
Rout out 0 1k
.op
.control
op
print v(in) v(out)
quit
.endc
.end
```

28.3 Python script

```
from __future__ import annotations

# ruff: noqa: E402, I001

import sys
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parents[1]))

from common import netlist_path, print_parsed_circuit, print_residuals

NETLIST = netlist_path(__file__)

print_parsed_circuit(NETLIST)
print_residuals(NETLIST)
```

28.4 Notes

The line `r_load in out Rload` is parsed as a resistor, while the component name remains `r_load`. This matters for stable reporting and for preserving the user's naming choices.

29 Example 26: Expected Parser Errors

The final runnable example demonstrates unsupported or malformed syntax. The `dnipro.cir` file is intentionally valid; the invalid cases are held inside the Python script so that the example itself can run successfully.

29.1 dnipro placeholder netlist

```
* This directory demonstrates expected parser errors in run.py.
* The dnipro.cir file itself is intentionally valid.
R1 in out 1k
```

29.2 ngspice placeholder template

```
* Valid placeholder wrapper for the invalid-syntax appendix example.
.param Vin=1
V1 in 0 DC {Vin}
R1 in out 1k
Rout out 0 1k
.op
.control
op
print v(in) v(out)
quit
.endc
.end
```

29.3 Python script

```

from __future__ import annotations

# ruff: noqa: E402, I001

import sys
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parents[1]))

from common import print_parse_errors

print_parse_errors(
    [
        "R-1 in out 1k",
        "1R in out 1k",
        "R.out in out 1k",
        "R1 in out R/2",
        "V1 in 0 SIN(0, Vm, omega)",
        ".tran 1m 1",
        ".include other.cir",
        "D1 out 0",
        ".model Ddefault D(Is=1e-14 n=1)",
    ]
)

```

29.4 Rejected syntax

The rejected cases include invalid component names, unsupported arithmetic in values, comma-separated source syntax, unsupported directives, malformed diode lines, and incomplete diode model definitions. These are parser/API boundaries, not simulator failures.

A Complete Runnable Example Catalog

All examples below are present in `examples/user_guide`. Each directory has the same three-file structure:

- `dnipro.cir`;
- `ngspice.cir`;
- `run.py`.

ID	Directory	Demonstrated feature
01	01_voltage_divider	Basic resistor divider, residuals, and classical $Ax = b$.
02	02_current_source_direction	SPICE current-source direction and sign of node voltage.
03	03_rc_time_descriptor	RC time-domain residuals and $Gx + C\dot{x} = b$.
04	04_rc_laplace	RC Laplace-domain $A(s)x = b$ and frequency substitution.
05	05_rl_branch_current	Inductor branch-current formulation in time and Laplace domains.
06	06_rlc_lowpass	Mixed RLC dynamics with capacitor derivative and inductor branch current.

ID	Directory	Demonstrated feature
07	07_dependent_sources	VCVS, VCCS, CCCS, and CCVS in a complete circuit.
08	08_diode_nonlinear	Nonlinear diode residual and expected linear-extraction rejection.
09	09_voltage_source_direction	Reversed voltage-source terminals and sign convention.
10	10_floating_capacitor	Capacitor derivative contribution to two non-ground nodes.
11	11_rl_descriptor	Descriptor matrix for an inductor to ground.
12	12_sine_voltage_source	Symbolic sinusoidal voltage source.
13	13_sine_current_source	Symbolic sinusoidal current source and KCL sign.
14	14_pulse_source	PULSE source expansion to symbolic Piecewise.
15	15_pwl_source	PWL source expansion with exact rational times.
16	16_parser_rvi_comments_ground_alias	Comments, ground aliases, numeric values, and <code>DC(...)</code> syntax.
17	17_parser_cl_values	Capacitor and inductor parser values.
18	18_diode_model_values	Exact numeric parsing of diode model parameters.
19	19_dependent_source_parser_bundle	Parser bundle for E, G, F, and H source types.
20	20_diode_parser_symbolic_model	Symbolic diode model parsing and isolated diode residual.
21	21_single_capacitor_frequency	Isolated capacitor and $s = j2\pi f$ substitution.
22	22_numeric_substitution_missing_params	Strict missing-parameter error.
23	23_linear_extraction_rejects_diode	Expected nonlinear rejection from linear extraction.
24	24_descriptor_rejects_laplace	Expected descriptor rejection in Laplace domain.
25	25_case_preserved_component_name	Case-insensitive component type with preserved original name.
26	26_invalid_examples_expected_errors	Expected parser and model errors for invalid syntax.

B Verification Commands

The example catalog can be checked from the repository root with:

```
for f in examples/user_guide/*/run.py; do
  /home/konst/Documents/Projects/venv/bin/python "$f"
done
```

```
for f in examples/user_guide/*/ngspice.cir; do
  ngspice -b "$f"
done
```

```
/home/konst/Documents/Projects/venv/bin/ruff check examples/user_guide
```